

File Systems

Main Points

- File system:
 - Useful abstractions on top of physical devices
 - Usage patterns
 - File layout
 - Directory layout

File Systems

- Abstraction on top of persistent storage
- Devices provide
 - Storage that (usually) survives across machine crashes
 - Block level (random) access
 - Large capacity at low cost
 - Relatively slow performance
 - Magnetic disk read takes 10-20M processor instructions

File System as Illusionist:

Hide Limitations of Physical Storage

- Persistence of data stored in file system:
 - Even if crash happens during an update
 - Even if disk block becomes corrupted
 - Even if flash memory wears out
- Naming:
 - Named data instead of disk block numbers
 - Directories instead of flat storage
 - Byte addressable data even though devices are block-oriented
- Performance:
 - Cached data
 - Data placement and data structure organization
- Controlled access to shared data

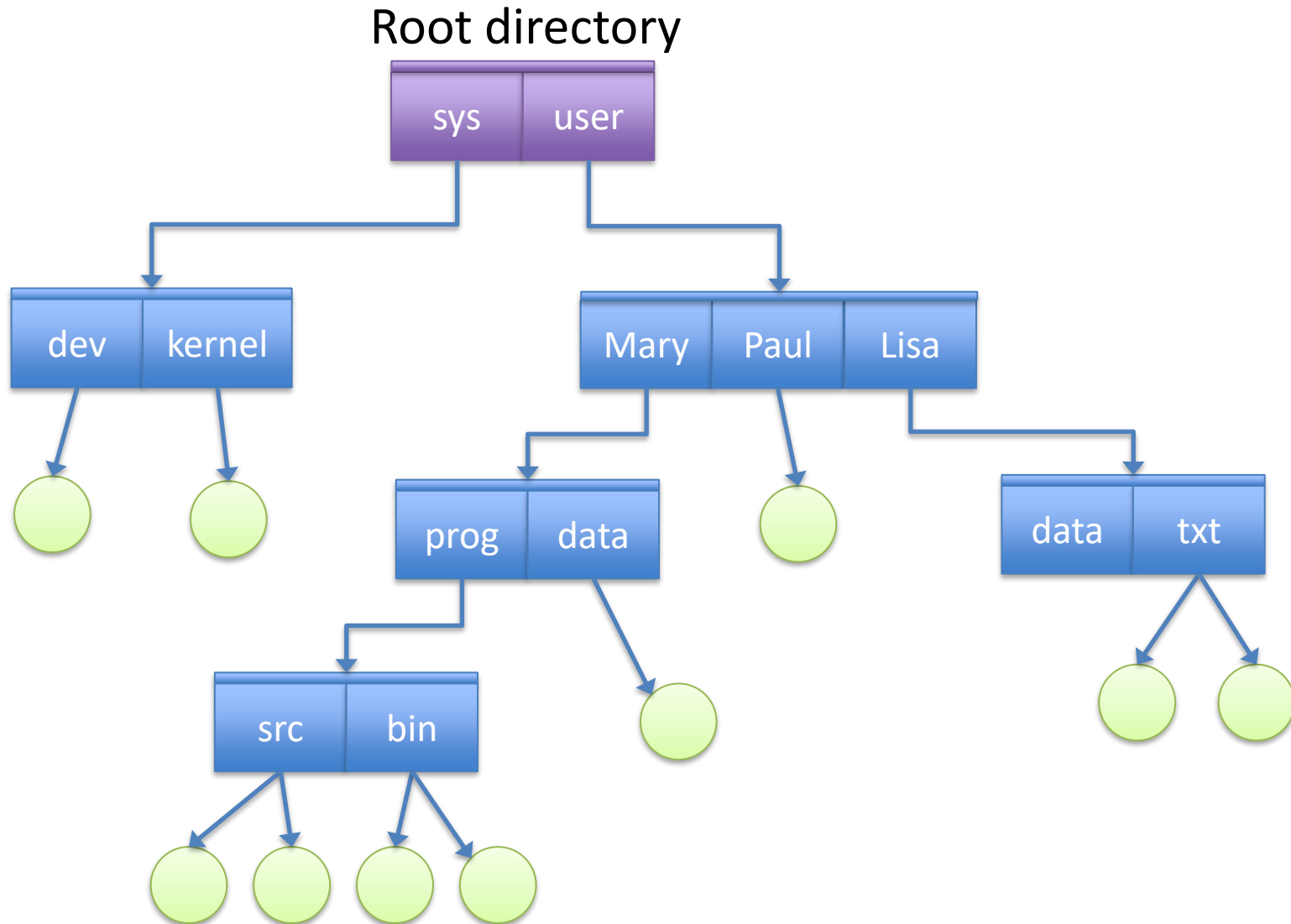
File System Abstraction

- File system
 - Hierarchical organization (directories, subdirectories)
 - Access control on data
 - File: named collection of data
 - Linear sequence of bytes (or a set of sequences)
 - Read/write or memory mapped
 - Crash and storage error tolerance
 - Operating system crashes (and disk errors) leave file system in a valid state
 - Performance
 - Achieve close to the hardware limit* in the average case
- (*) considering both the device and the memory subsystem

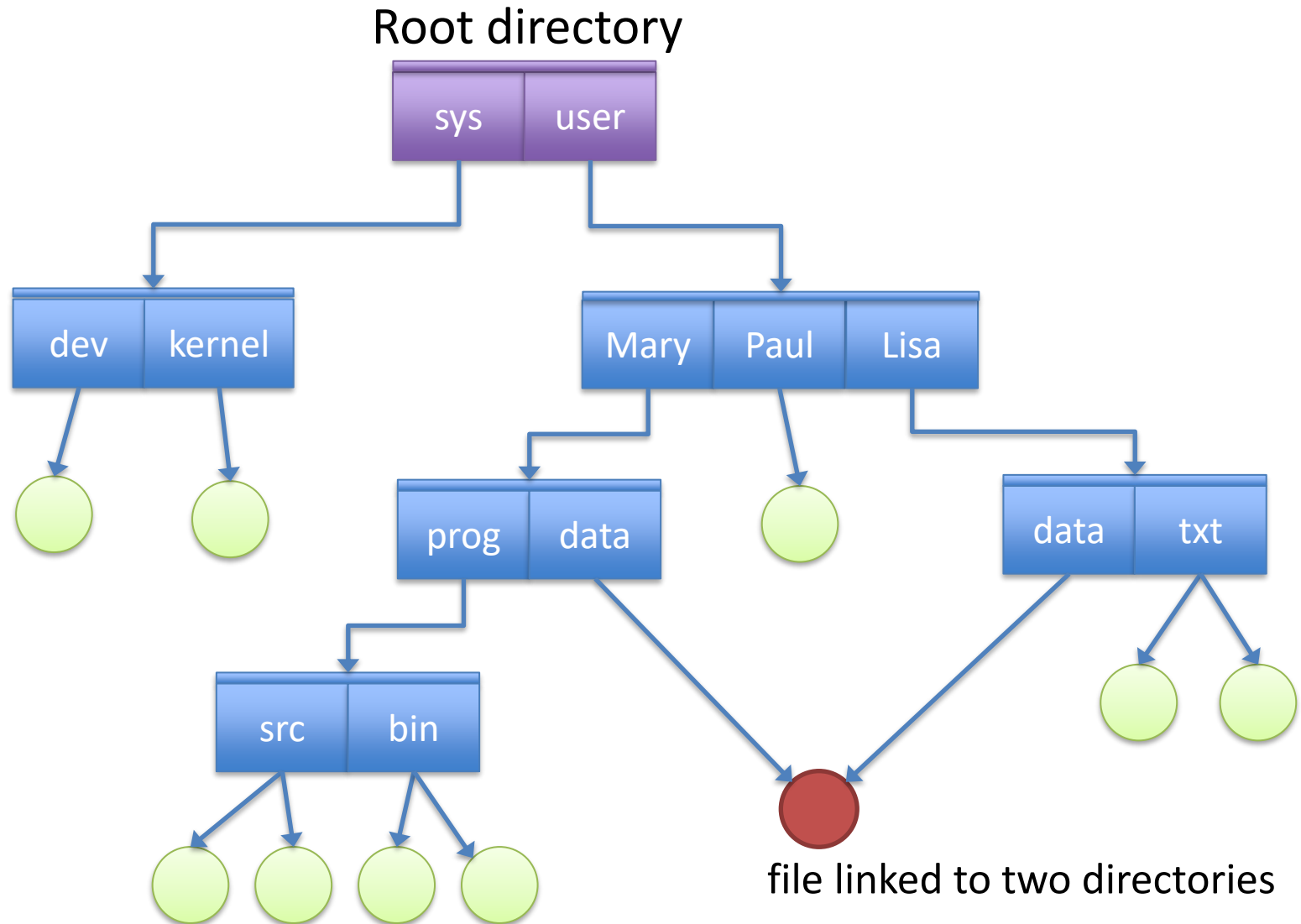
File System Abstraction

- Directory
 - Group of named files or subdirectories
 - Mapping from file name to file metadata location
- Path
 - String that uniquely identifies file or directory
 - Ex: /cse/www/education/courses/cse451/12au
- Links
 - Hard link: link from name to metadata location
 - Soft link: link from name to alternate name
- Mount
 - Mapping from name in one file system to root of another

File System: tree structure



File System: acyclic graph structure

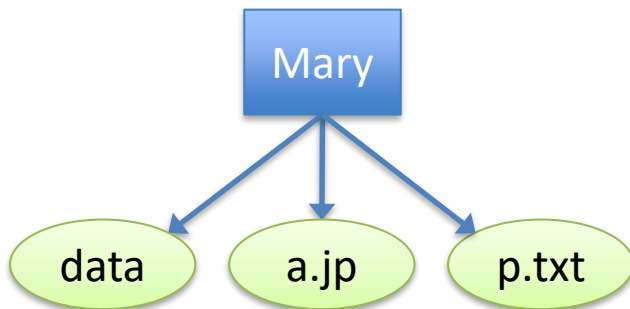


Data structures: Directories

- Each file (and directory) belongs to a directory;
- From the logical standpoint, each directory is a data structure that links file names to file attributes (file attributes might not be stored in the directory)
 - Example of attributes: file size, address on disk, access rights, time of last access, time of creation, ...

A possible implementation (FAT file systems):

- The directory is a table, it associates each file name to its file descriptor (which includes all attributes of the file)



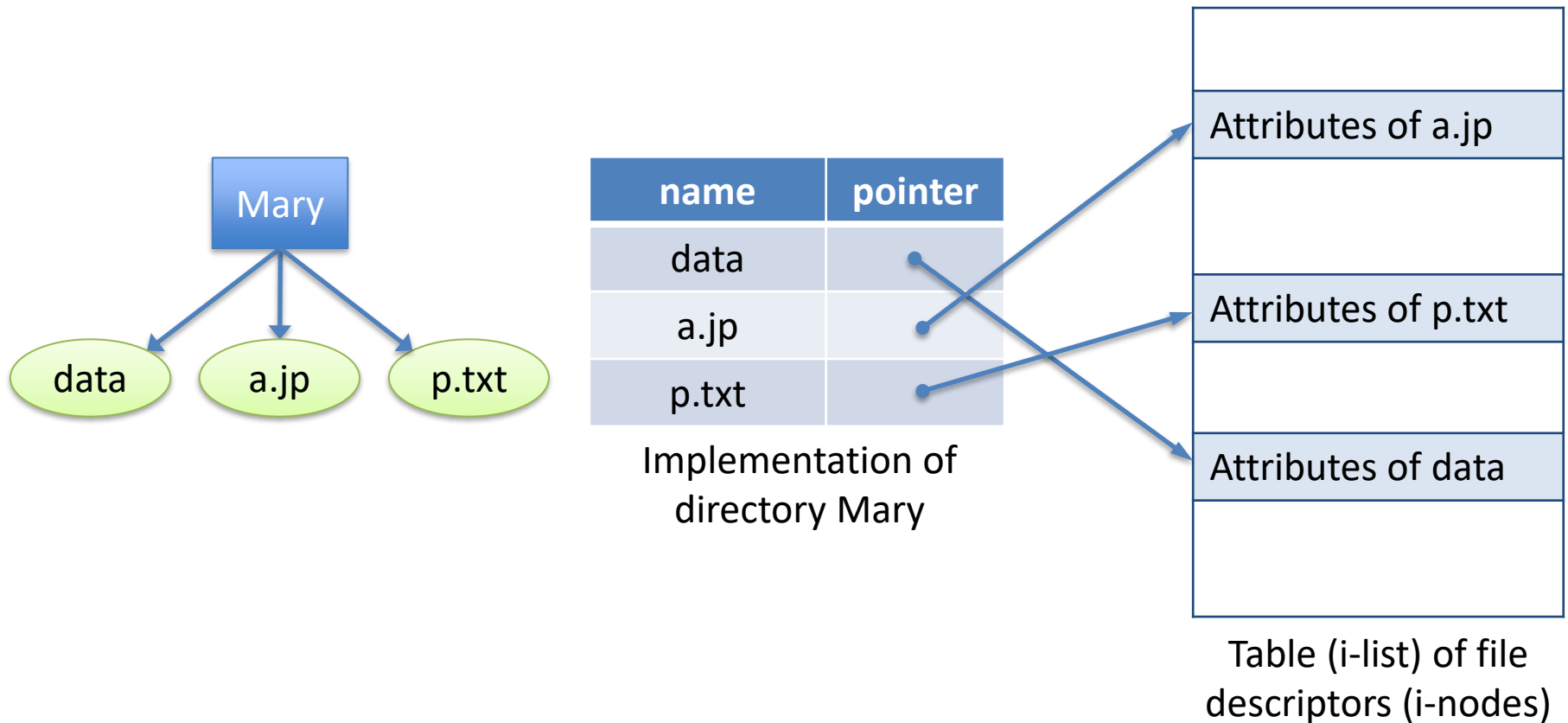
name	descriptor
data	Attribute: type, address, etc.
a.jp	Attribute: type, address, etc.
p.txt	Attribute: type, address, etc.

Implementation of directory Mary

Data structures: Directories

Implementation of directories in Unix:

- The directory is a table that includes the references to the file descriptors (i-nodes), which are stored in a separate data structure in the disk



Access to files

- File access operations:
 - Read a logical record from a file.
 - Write logical records on a file

To execute these operations on a file it is necessary to retrieve the attributes of the file:

- ▣ The addresses on disk of the logical records
- ▣ Access rights
- ▣ etc.....

Too expensive to read these data from the disk at each access:

- they are read once for all before accessing the file, by means of the **open** SC
- the **close** SC is then necessary to deallocate such information from memory

Access methods

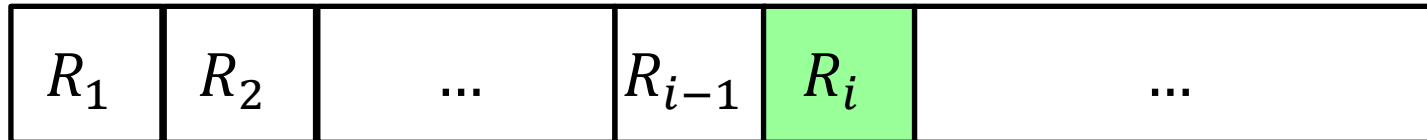
- File access can be:
 - Sequential
 - Direct

The access method:

- independent of the physical device
- independent of the allocation method of the files on the device

Sequential access

- The file is a sequence of logical records $[R_1, R_2, \dots, R_N]$:
 - To access each logical record R_i it is necessary to access first all the previous $(i - 1)$ records:



- Access operations:
 - *readnext*: read next logical record
 - *writenext*: write next logical record
- Each operation (read/write) positions an access pointer on the next logical record

Direct Access

- The file is a set $\{R_1, R_2, \dots R_N\}$ of logical records
 - The user can directly access each logical record
 - Access operations:
 - *read(f, i, &V)*: read logical record i of file f ; the read data is stored in buffer V ;
 - *write(f, i, &V)*: write the content of buffer V on the logical record i of file f .

UNIX File System API

- create, link, unlink, mkdir, rmdir
 - Create file, link to file, remove link
 - Create directory, remove directory
- open, close, read, write, seek (mmap, munmap)
 - Open/close a file for reading/writing
 - Seek resets current position
- fsync
 - File modifications can be cached
 - fsync forces modifications to disk (like a memory barrier)

File System Interface

- UNIX file open is a Swiss army knife:
 - Open the file, return file descriptor
 - Options:
 - if file doesn't exist, return an error
 - If file doesn't exist, create file and open it
 - If file does exist, return an error
 - If file does exist, open file
 - If file exists but isn't empty, nix it then open
 - If file exists but isn't empty, return an error
 - ...

Interface Design Question

- Why not separate syscalls for open/create/exists?
 - Would be more modular!
 - But rather boring as well...

```
if (!exists(name))
```

```
    create(name); // can create fail?
```

```
fd = open(name); // does the file exist?
```

File System Workload

- File sizes
 - Are most files small or large?
 - Which accounts for more total storage: small or large files?

File System Workload

- File sizes
 - Are most files small or large?
 - SMALL
 - Which accounts for more total storage: small or large files?
 - LARGE

File System Workload

- File access
 - Are most accesses to small or large files?
 - Which accounts for more total I/O bytes: small or large files?

File System Workload

- File access
 - Are most accesses to small or large files?
 - SMALL
 - Which accounts for more total I/O bytes: small or large files?
 - LARGE

File System Workload

- How are files used?
 - Most files are read/written sequentially
 - Ex. Image files, executables
 - Some files are read/written randomly
 - Ex: database files, swap files
 - Some files have a pre-defined size at creation
 - Some files start small and grow over time
 - Ex: program stdout, system logs

File System Design

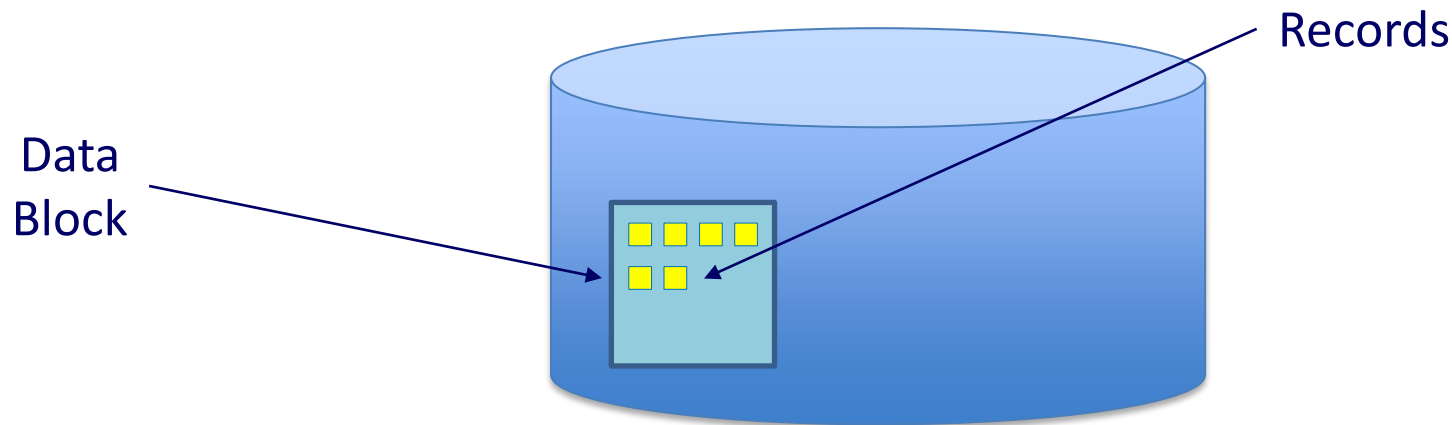
- For small files:
 - Small blocks for storage efficiency
 - Files used together should be stored together
- For large files:
 - Large blocks for storage efficiency
 - Contiguous allocation for sequential access
 - Efficient lookup for random access
- May not know at file creation
 - Whether file will become small or large
 - Whether file is persistent or temporary
 - Whether file will be used sequentially or randomly

File System Design

- Data structures
 - **Directories:** file name -> file metadata
 - Store directories as files
 - **File metadata:** how to find file data blocks + other properties of files
 - **Free map:** list of free disk blocks
- How do we organize these data structures?
 - Device has non-uniform performance

Data blocks and records

- Data in files are accessible in records
 - e.g., in Unix a record is a single byte
- Data are physically stored (and accessed) in blocks
- Usually block size $\gg$ record size



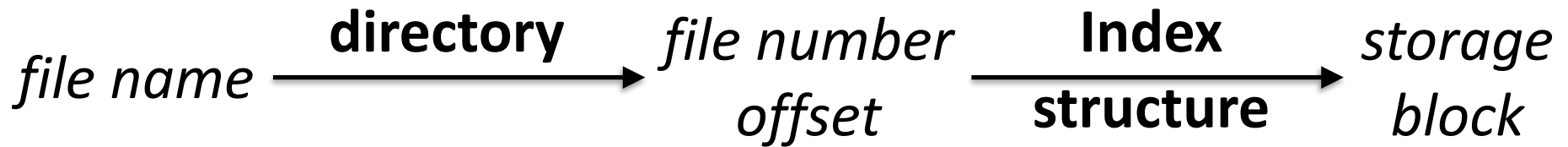
Design Challenges

- Index structure
 - How do we locate the blocks of a file?
- Index granularity
 - What block size do we use?
- Free space
 - How do we find unused blocks on disk?
- Locality
 - How do we preserve spatial locality?
- Reliability
 - What if machine crashes in middle of a file system op?

File System Design Options

	FAT	FFS	NTFS
Index structure	Linked list	Tree (fixed, asym.)	Tree (dynamic)
granularity	block	block	extent
free space allocation	FAT array	Bitmap (fixed location)	Bitmap (file)
Locality	defragmentation	Block groups + reserve space	Extents Best fit defrag

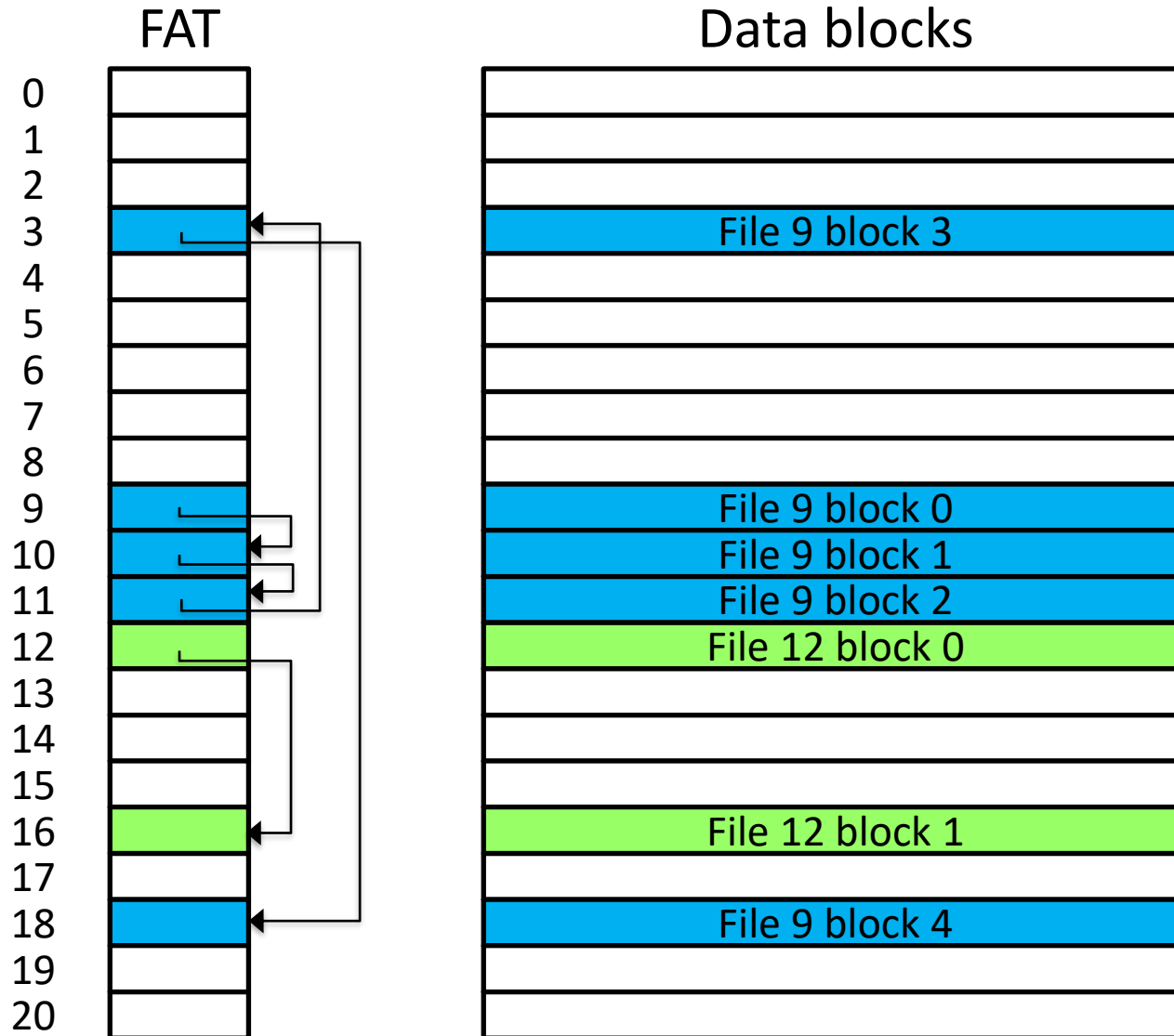
Named Data in a File System



Microsoft File Allocation Table (FAT)

- Linked list index structure
 - Simple, easy to implement
 - Still widely used (e.g., thumb drives)
- File table:
 - Linear map of all blocks on disk
 - Each file a linked list of blocks

FAT file system



FAT

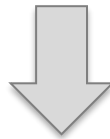
- Pros:
 - Easy to find free block
 - Easy to append to a file
 - Easy to delete a file
- Cons:
 - FAT size
 - Should be uploaded in main memory
 - Limitation to file system size
 - Limited metadata and no protection
 - Random access is slow
 - Fragmentation
 - File blocks for a given file may be scattered
 - Files in the same directory may be scattered
 - Problem becomes worse as disk fills

Limitazioni del file system FAT

Posto:

- L lunghezza (in bit) degli elementi della FAT
- B la dimensione(in byte) dei blocchi del disco

Il numero di blocchi indirizzabili è 2^L (capacità del disco, o partizione)



la massima estensione del file system è:

2^L blocchi, ovvero a $B \cdot 2^L$ byte.

se ogni elemento occupa N byte (solitamente L è multipla del byte), la FAT occupa complessivamente: $N \cdot 2^L$ byte

Limitazioni del file system FAT

Esempio: con $N=2$ ($\rightarrow$ FAT 16) e $B=2^{10}$ (blocchi di 1 Kbyte)

La massima estensione del file system è 2^{16} blocchi (2^{26} byte = 64MB)

La FAT occupa complessivamente $2 * 2^{16}$ byte = 128 Kbyte

- Sul disco la FAT occupa 128 KB, cioè 128 blocchi.
- con una memoria paginata e pagine di 1Kbyte, la FAT occupa 128 pagine in memoria principale.

Dato che gli elementi che descrivono un file possono essere distribuiti su molte pagine diverse, possono verificarsi frequenti errori di pagina quando si percorre un file.

$\rightarrow$ Per realizzare file systems più estesi si usano blocchi di dimensioni maggiori.

Limitazioni dei file systems FAT

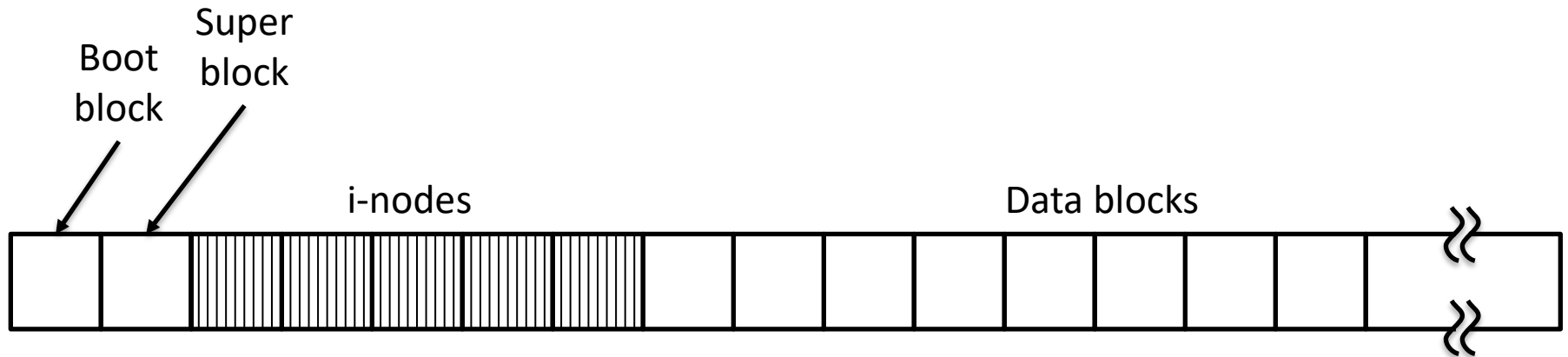
Block size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

- Massima dimensione del File System per diverse ampiezze dei blocchi

Berkeley UNIX FFS (Fast File System)

- inode table (i-list)
 - “Analogous” to FAT table, but does much more...
- inode
 - Metadata
 - File owner, access permissions, access times, ...
 - Set of pointers to data blocks

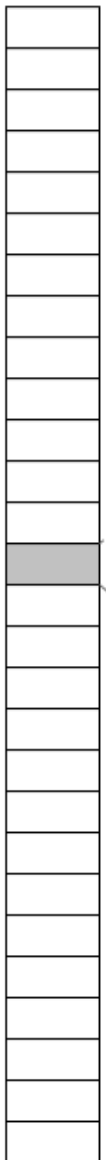
Physical disk organization in UNIX



FFS inode

- Metadata
 - File owner, access permissions, access times, ...
- Set of 12 data pointers (of 32 bits each)
 - With 4KB blocks => max size of 48KB
- Indirect block pointer
 - pointer to disk block of data pointers
 - 4KB block size => 1K pointers to data blocks => 4MB
- Doubly indirect block pointer
 - Doubly indirect block => 1K indirect blocks
 - 4GB (+ 4MB + 48KB)
- Triply indirect block pointer
 - Triply indirect block => 1K doubly indirect blocks
 - 4TB (+ 4GB + 4MB + 48KB)

Inode Array



File
Metadata

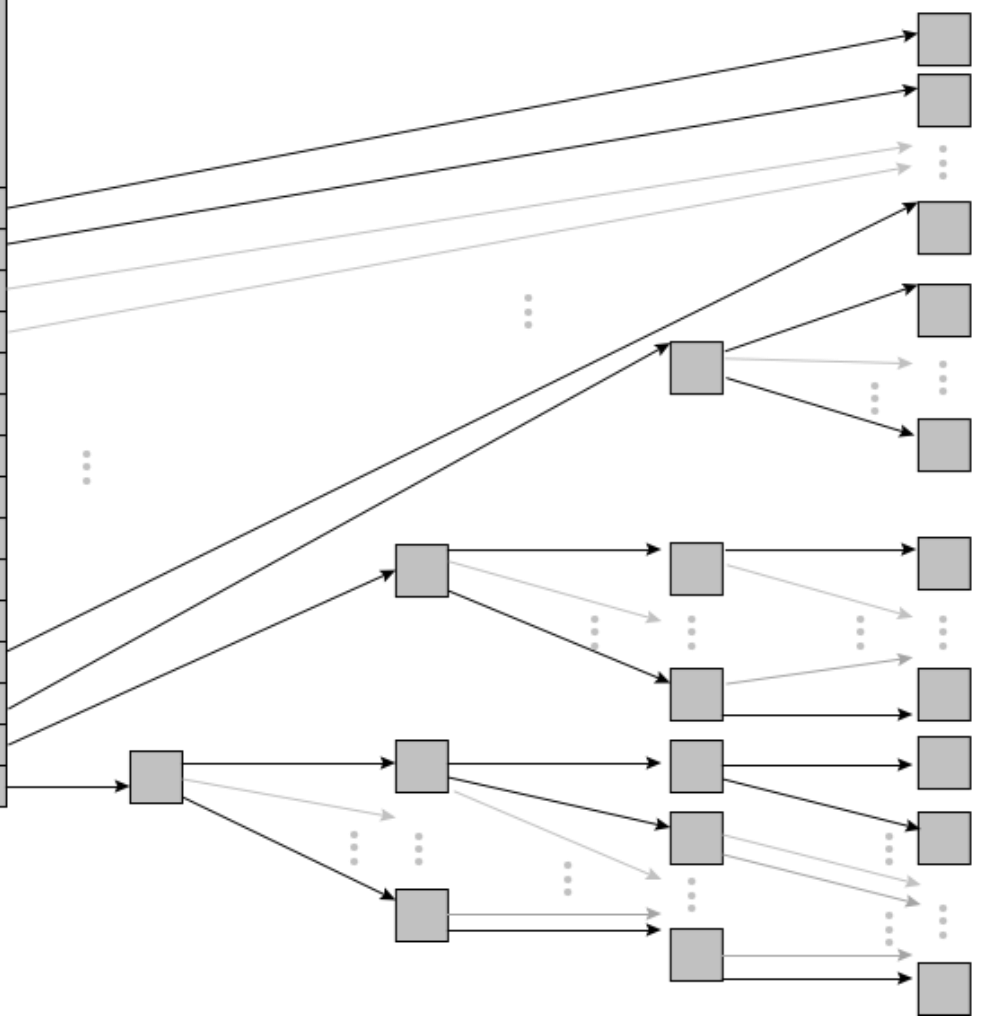
Direct
Pointers

Indirect Pointer
Dbl. Indirect Ptr.
Tripl. Indrect Ptr.

Inode



Triple Indirect Blocks Double Indirect Blocks Indirect Blocks Data Blocks

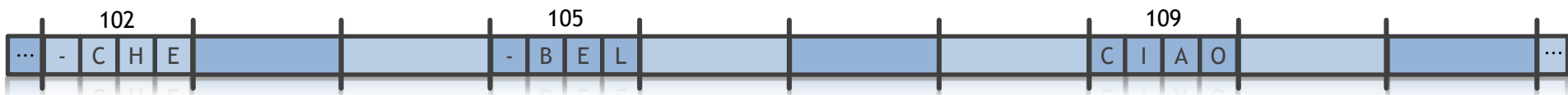


Esempio (semplificato) di i-node

- Blocchi di 4 byte
- Puntatori di 2 byte
- File di 30 byte (quindi 8 blocchi)
- I node con:
 - 2 puntatori diretti
 - 1 puntatore indiretto singolo
 - 1 puntatore indiretto doppio

Esempio (semplificato) di i-node

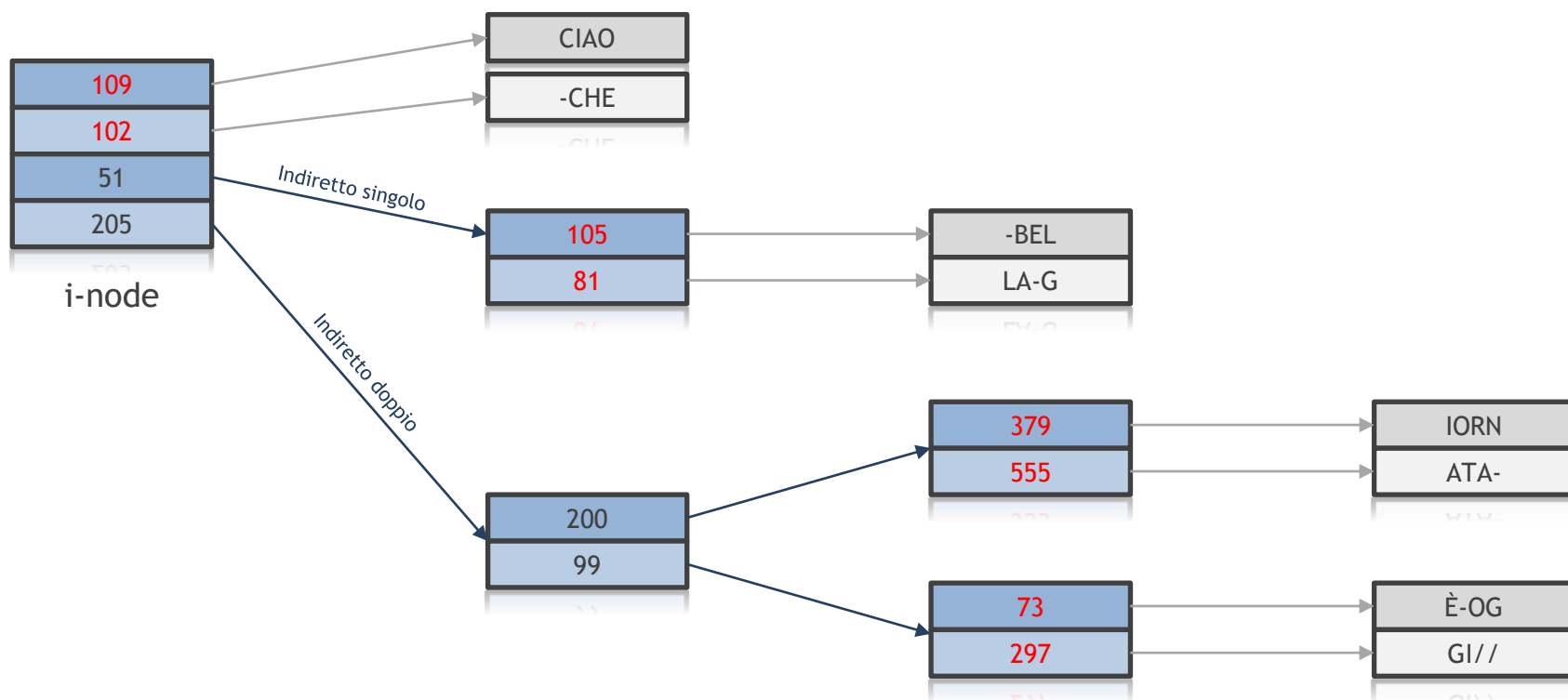
Supponiamo di avere il seguente file
Suddiviso in blocchi da 4 byte ciascuno:



Ogni blocco **logico** del file corrisponde
Ad un blocco **fisico** nel disco

Esempio (semplificato) di i-node

Supponiamo adesso che gli indirizzi siano a 2 byte



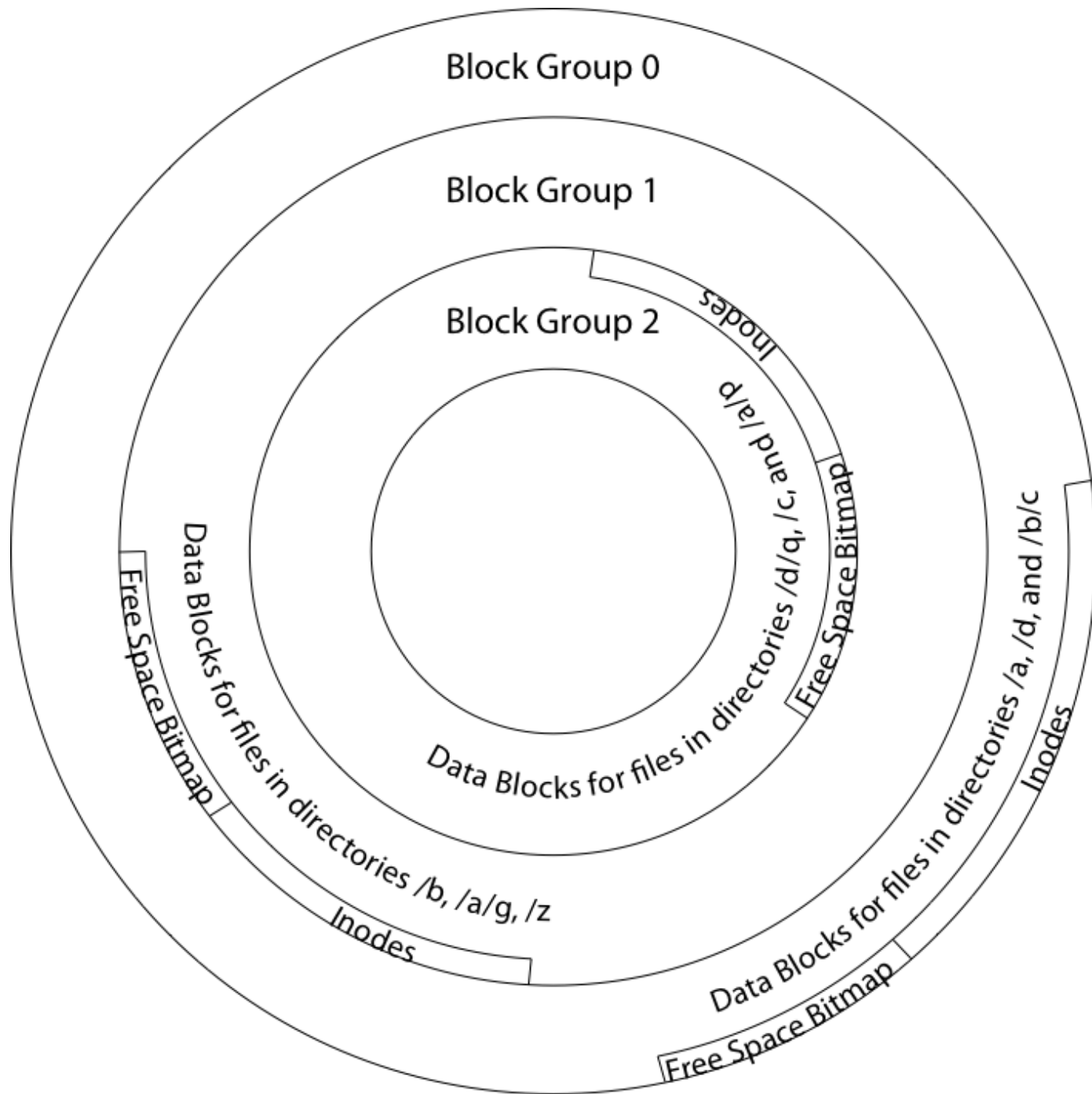
Quanti blocchi sono indirizzabili da ogni puntatore?
 $4\text{Byte}/2\text{Byte} = 2$

FFS Asymmetric Tree

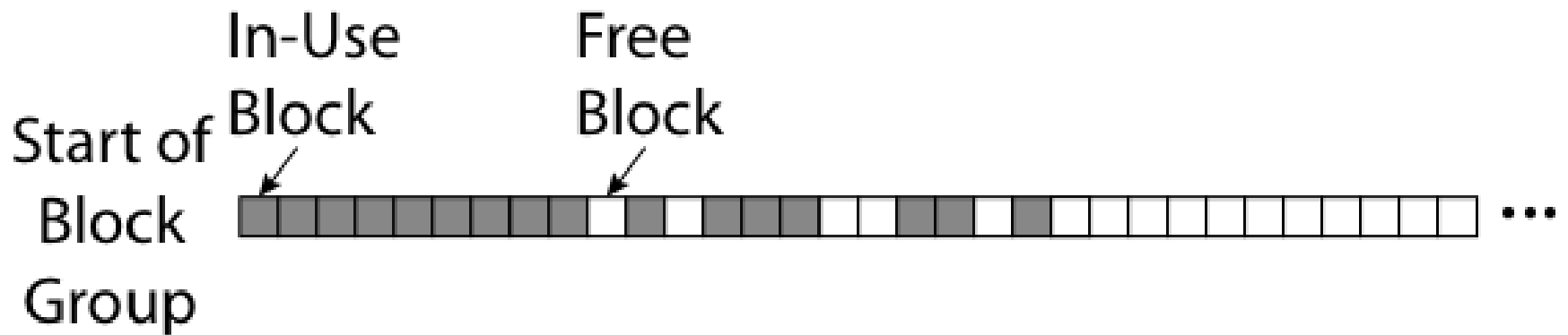
- Small files: shallow tree
 - Efficient storage for small files
- Large files: deep tree
 - Efficient lookup for random access in large files
- Sparse files: only fill pointers if needed

FFS Locality

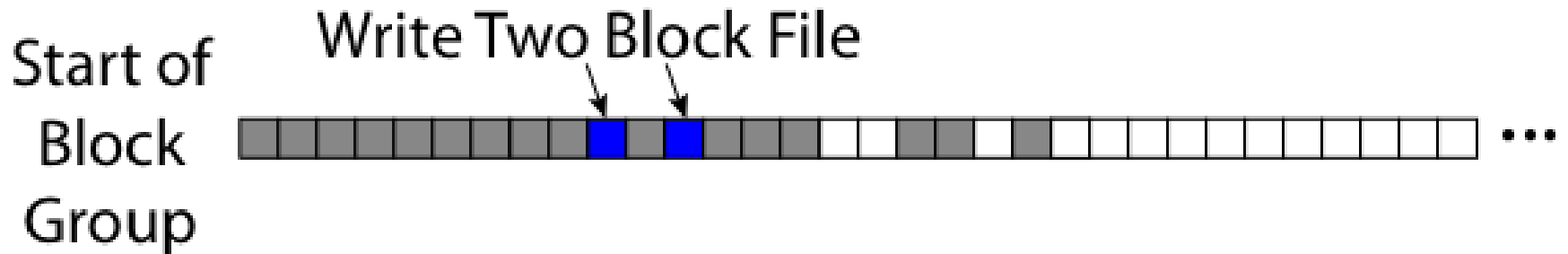
- Block group allocation
 - Block group is a set of nearby cylinders
 - Files in same directory located in same group
 - Subdirectories located in different block groups
- inode table spread throughout disk
 - inodes, bitmap near file blocks
- First fit allocation
 - Small files fragmented; large files contiguous (if disk not close to full utilization...)



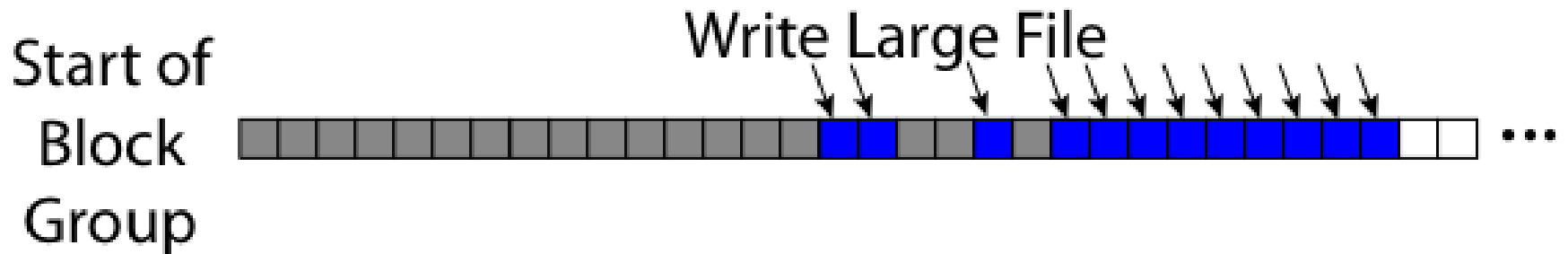
FFS First Fit Block Allocation



FFS First Free Block Allocation



FFS First Free Block Allocation



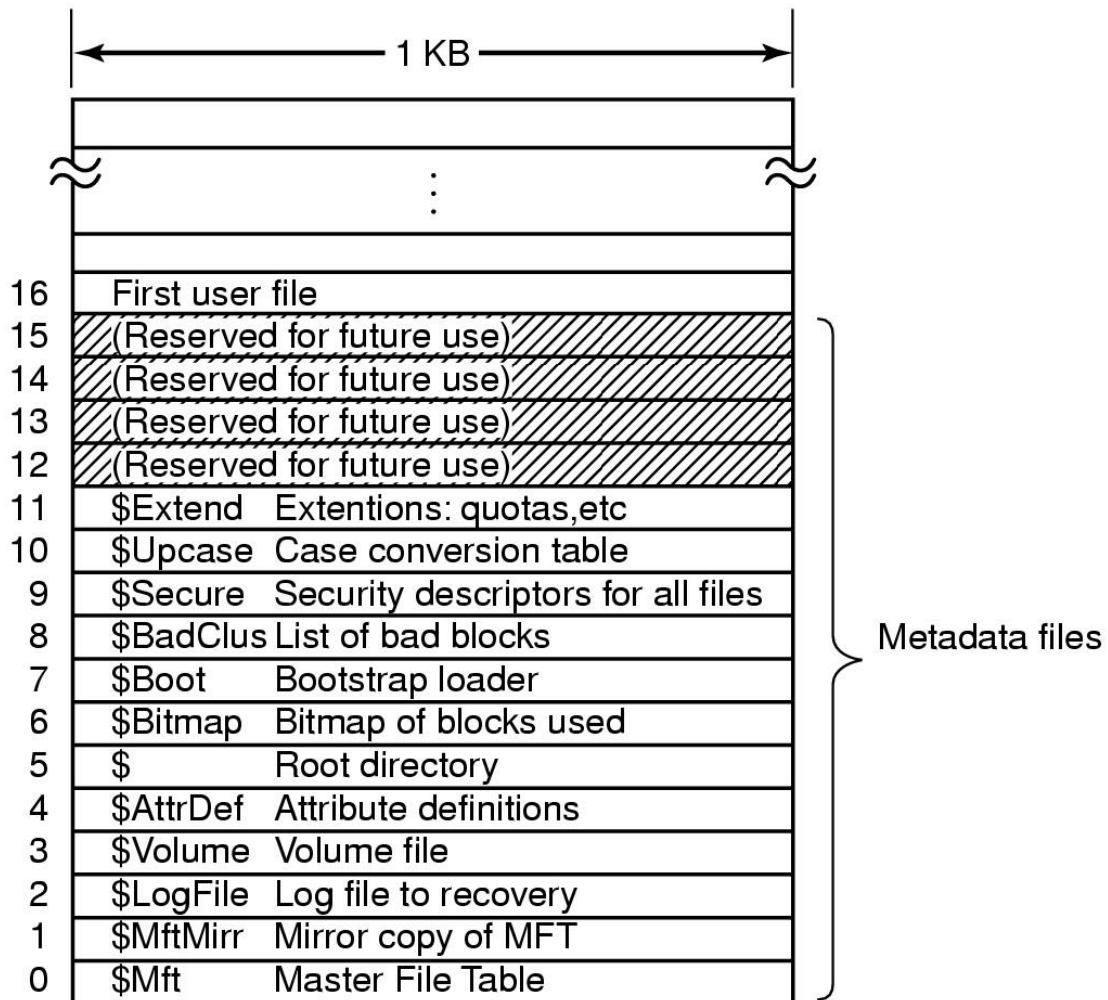
FFS

- Pros
 - Efficient storage for both small and large files
 - Locality for both small and large files
 - Locality for metadata and data
- Cons
 - Inefficient for tiny files (a 1 byte file requires both an i-node and a data block)
 - Inefficient encoding when file is mostly contiguous on disk (no equivalent to superpages)
 - Need to reserve 10-20% of free space to prevent fragmentation

NTFS

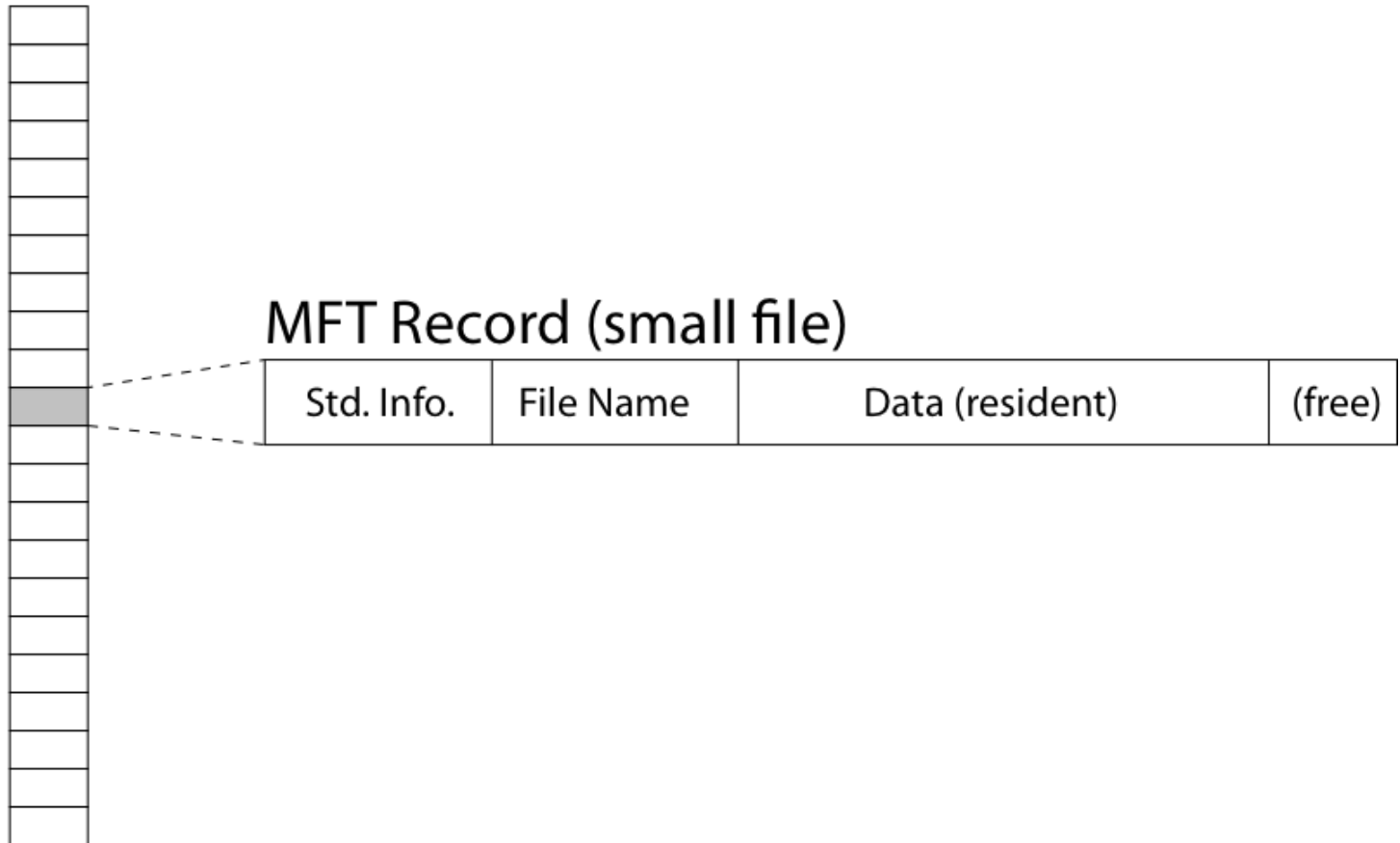
- Master File Table
 - A table of records (one x file)
 - Flexible 1KB storage for a file metadata and data
 - MFT itself is a file!
- Extents
 - Block pointers cover runs of blocks
 - Linux (ext4) adopts a similar approach
 - File create can provide hint as to size of file
- Journalling for reliability
 - Will not be discussed!

MFT

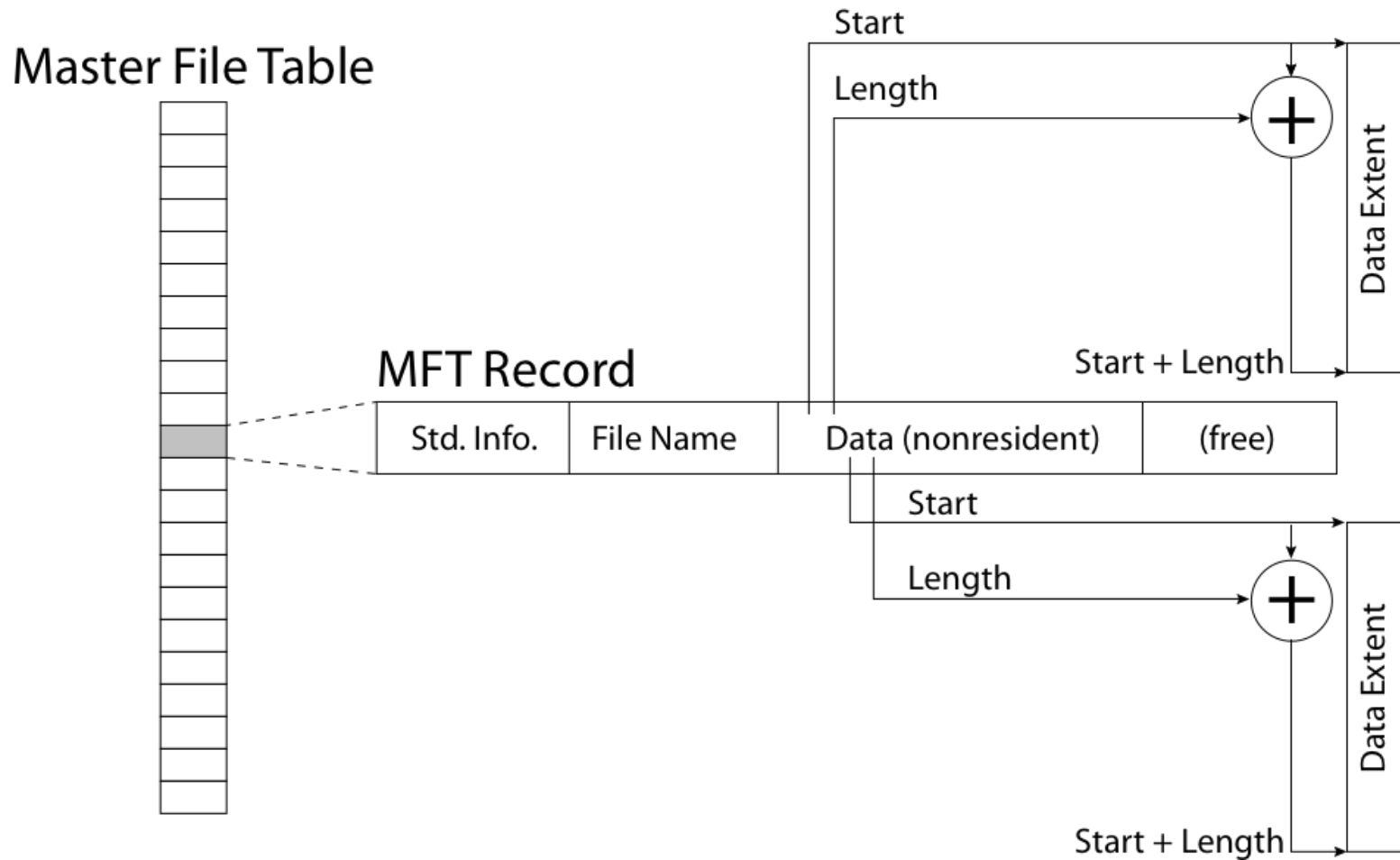


NTFS Small File

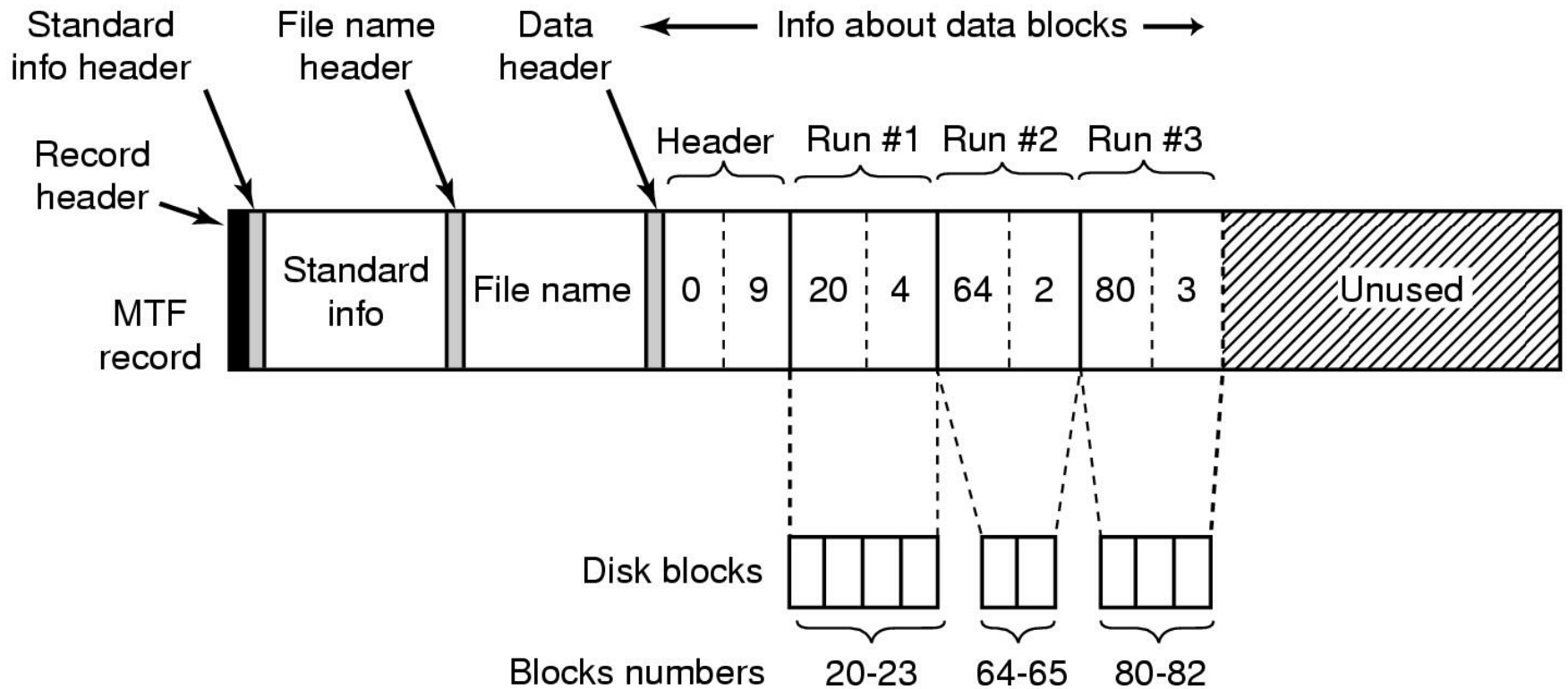
Master File Table



NTFS Medium File

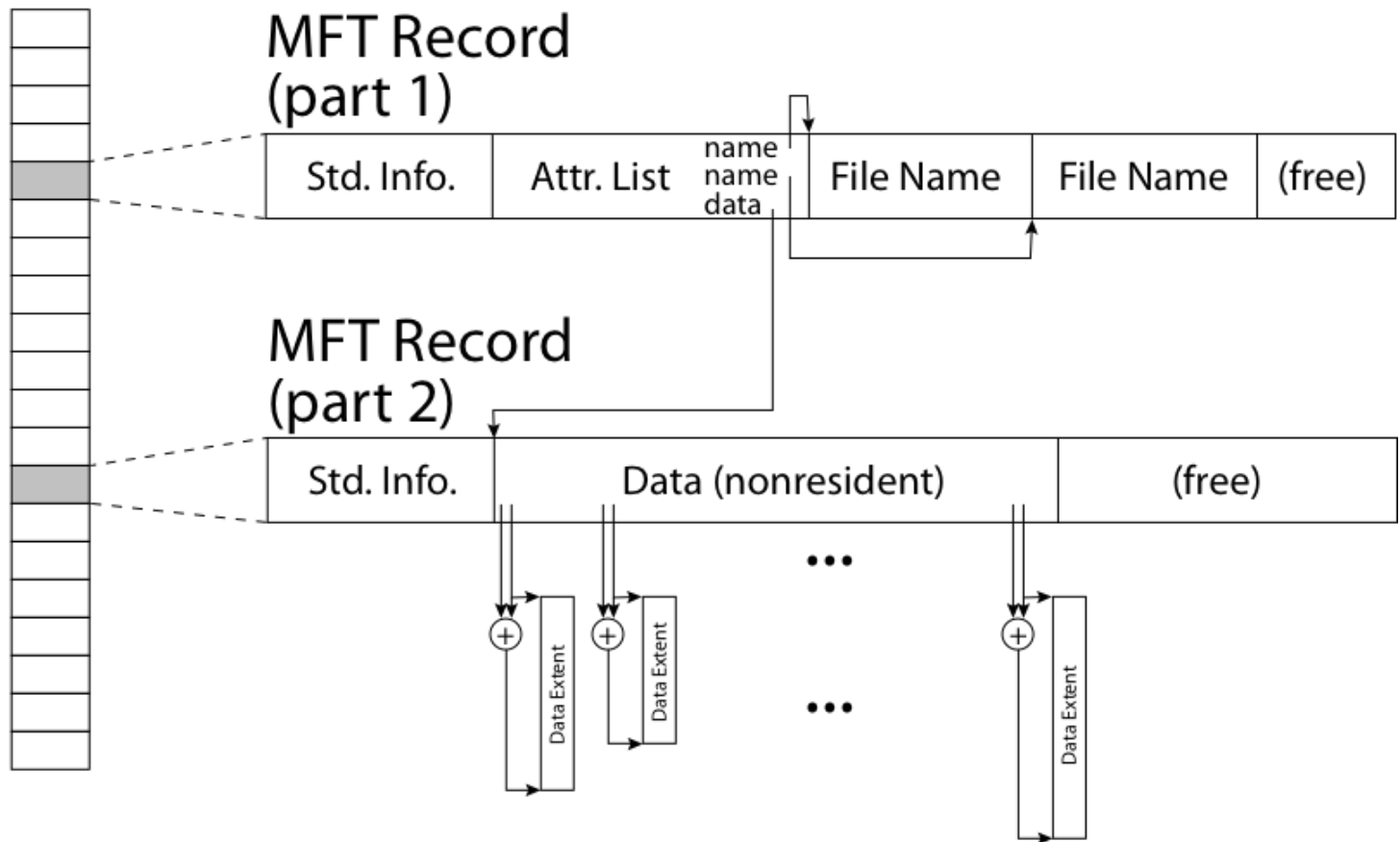


NTFS Medium File

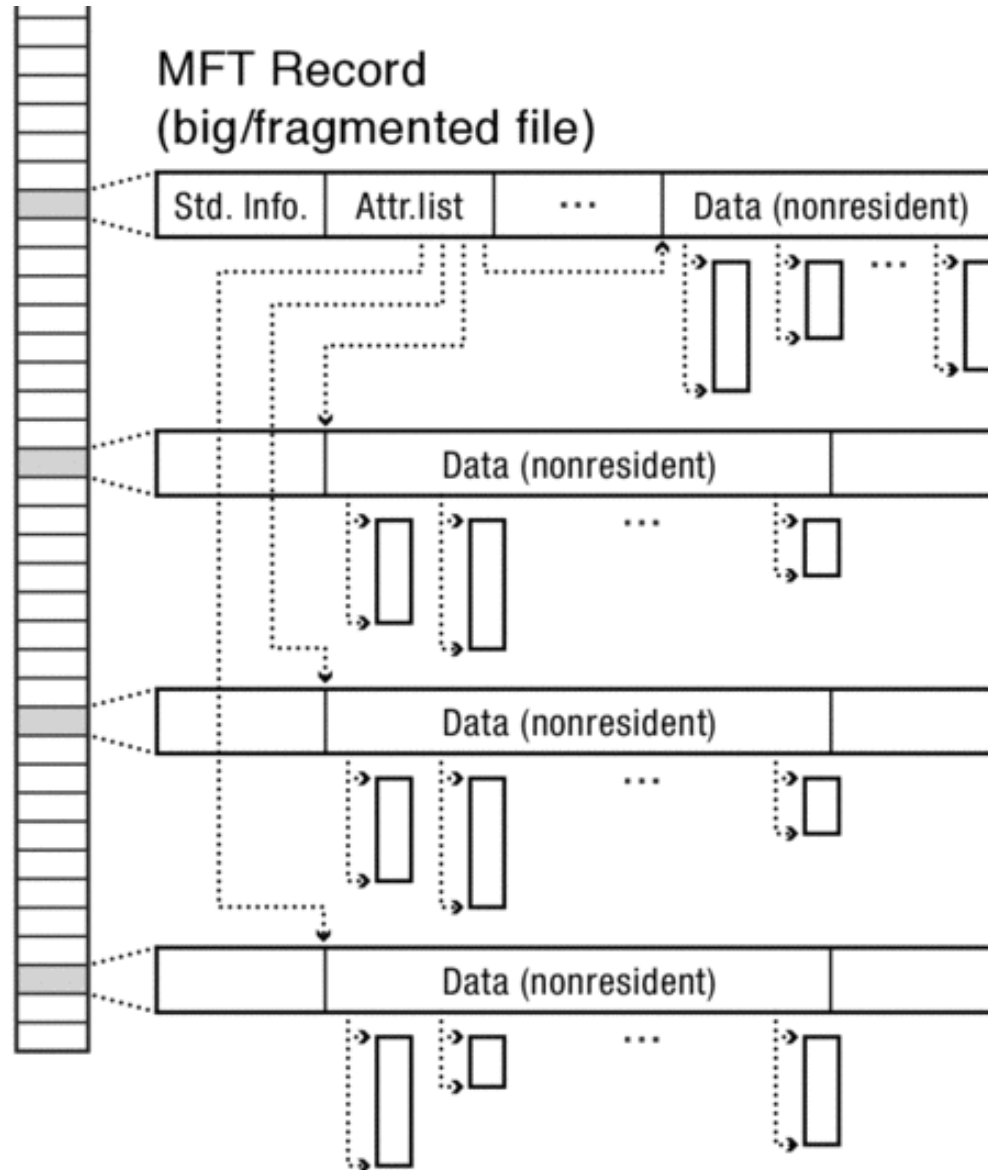


NTFS Indirect Block

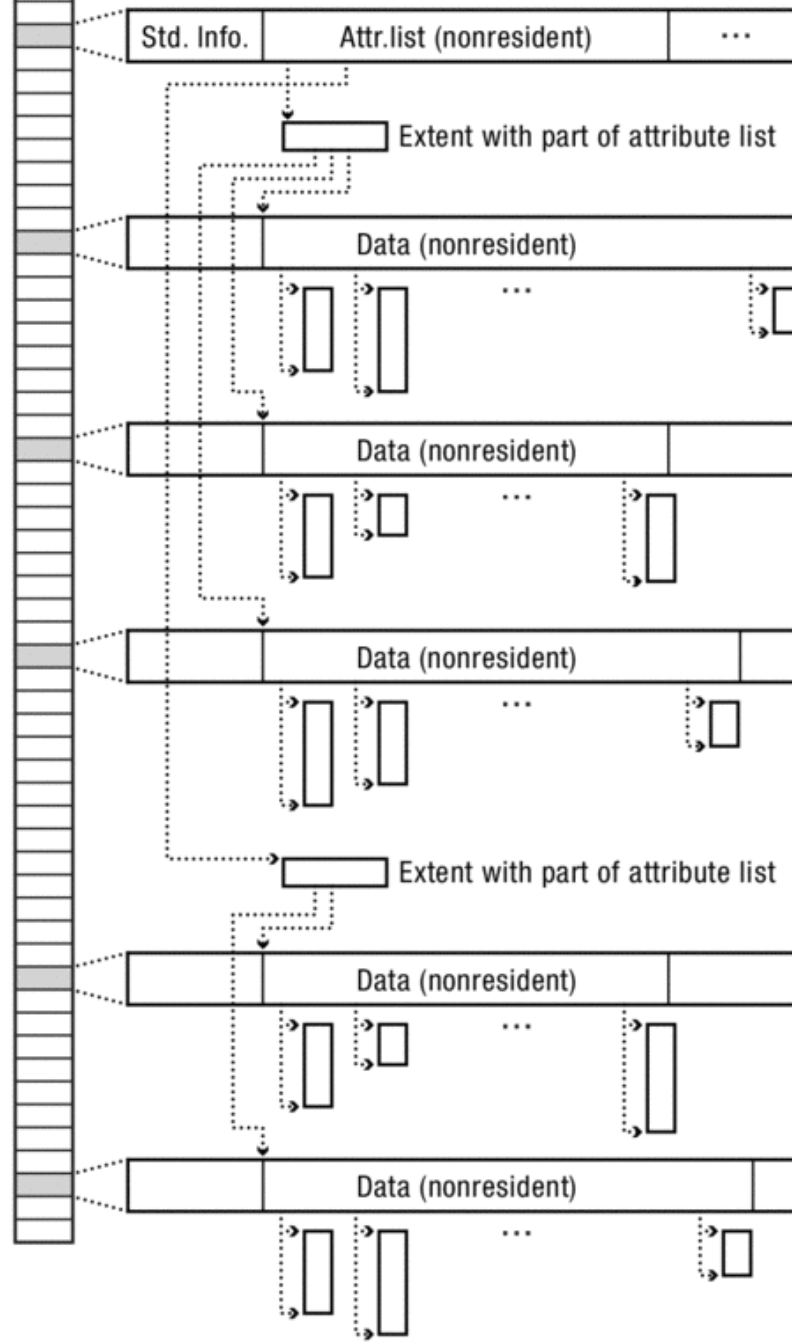
Master File Table



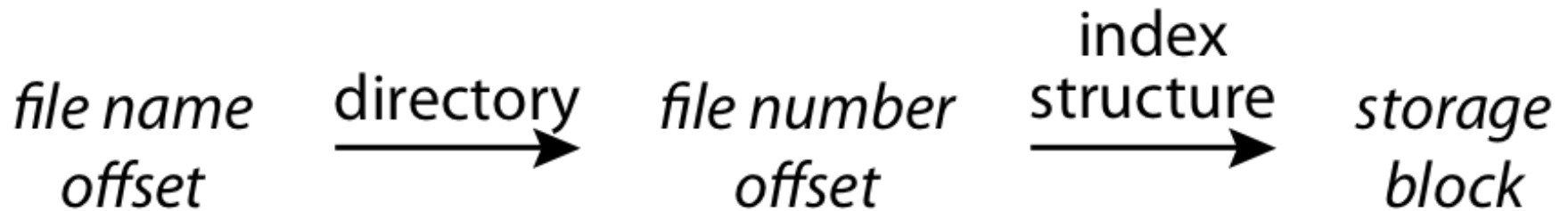
NTFS Multiple Indirect Blocks



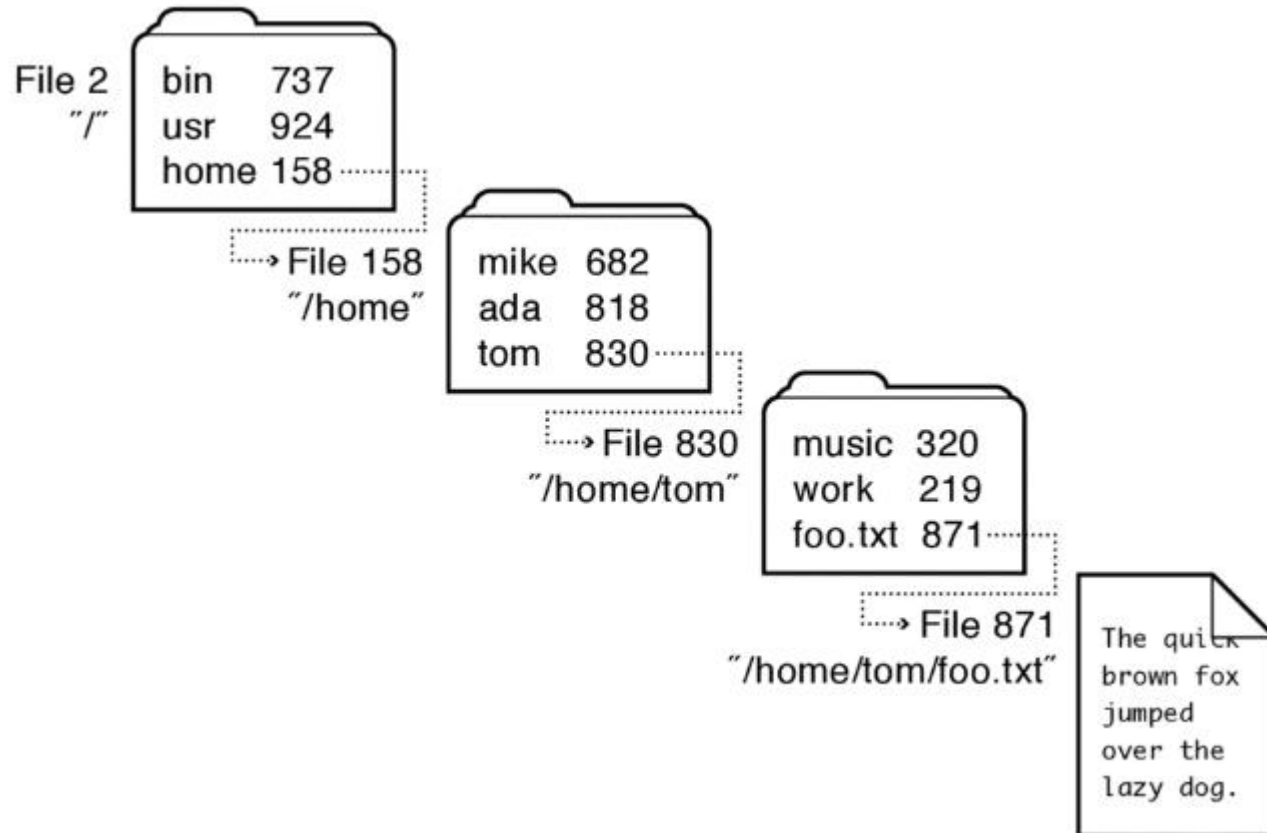
MFT Record (huge/badly-fragmented file)



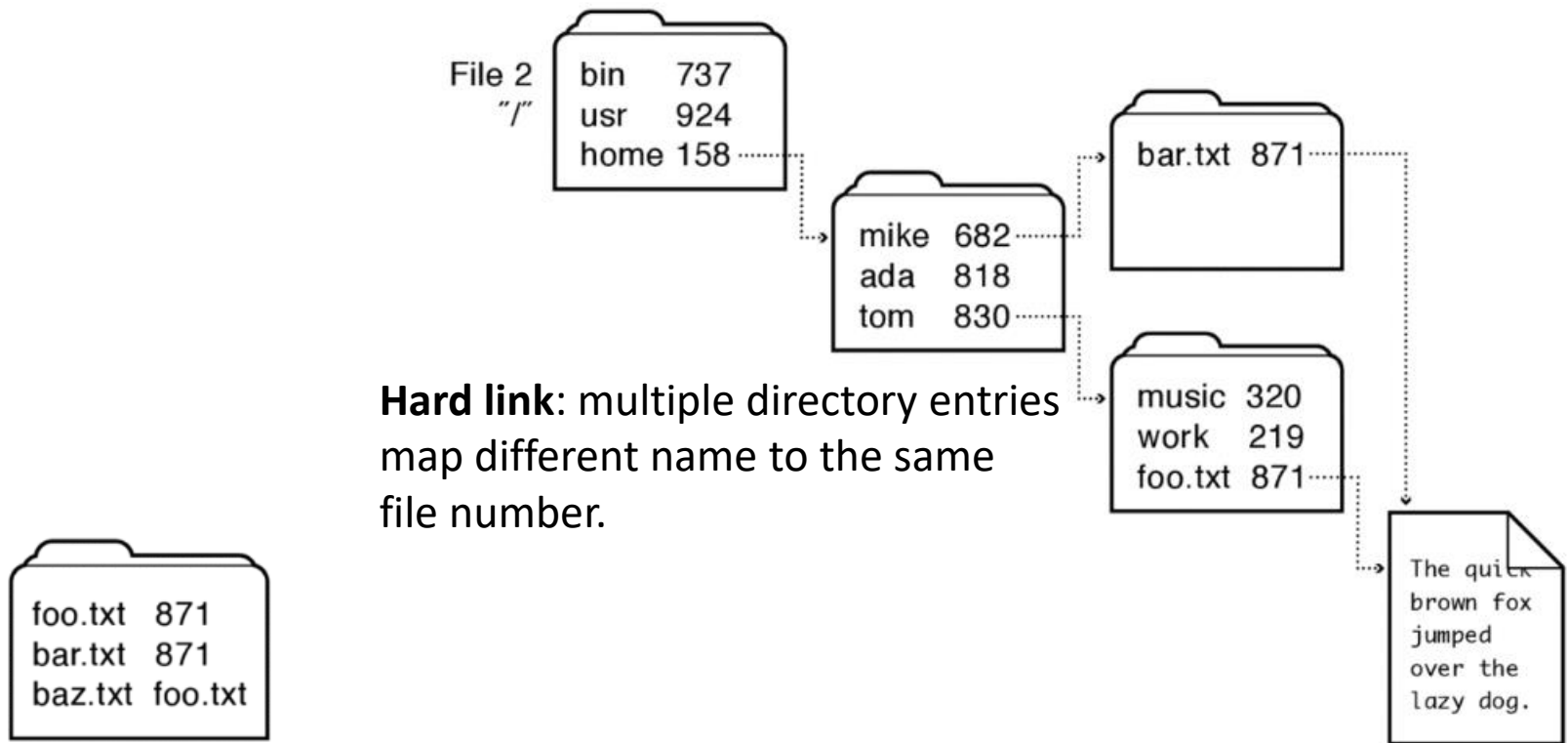
Named Data in a File System (remind)



Named Data in a File System



Hard & Soft link

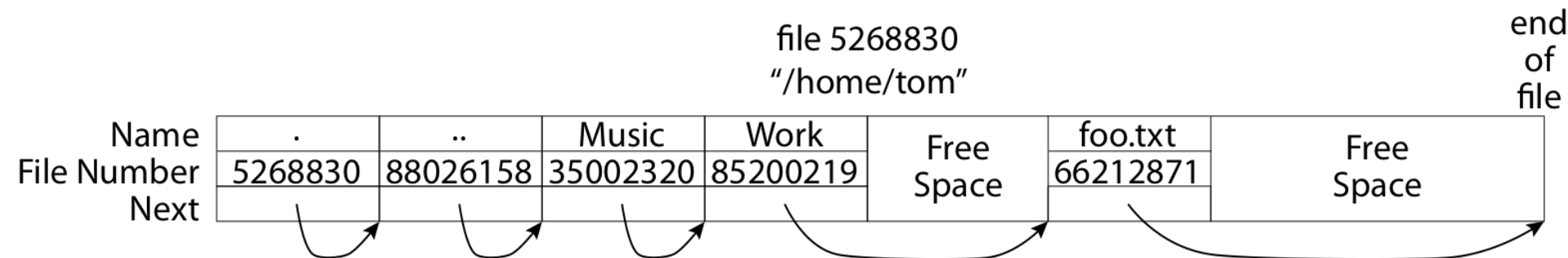


Hard link: multiple directory entries map different name to the same file number.

Soft link: a directory entry that maps one name to another name

Directories

- Directories are files
 - Map file name to file number (MFT # or inode num)
- Table of file name -> file number
 - Small directories: linear search



Large Directories: B-Trees

Search for Hash (foo.txt) = 0x30

